

实验题目： 简单字符设备驱动程序

一、 实验内容

按实验内容编写驱动程序及测试程序

编译驱动程序和应用程序

在嵌入式设备上加载驱动程序并进行测试

要求写一个简单的字符设备驱动程序，要求：

定义一个数据结构，该数据结构中包含一个大小为 1024 的 buffer 和一个 count 整形变量；定义一个全局的该数据结构指针，初始化为 NULL；

在 open 中对该指针进行 NULL 判断，为 NULL 则为其分配内存，并将数据结构中的 buffer 初始化为 0，将 count 初始化为 1；如果该指针不为 NULL，则只将 count 加 1；

在 release 中，将 count 减 1；如果 count 为 0，则释放 open 中申请的空间，并把指针置为 NULL；

在 read 里面对该 buffer 进行读取；

在 write 里面对该 buffer 进行赋值（任意赋值即可）

写测试程序进行测试，可以一个进程读，一个进程写，实现进程间通信

注意事项：

在编写驱动程序及测试程序时，在关键位置增加打印信息，使得测试结果打印输出可以显示驱动程序加载卸载以及应用程序调用驱动程序的过程。

驱动程序打印调用 printk 函数

驱动程序申请空间调用 kmalloc 函数

在 ubuntu 虚拟机上完成验证

二、 实验原理（代码以及注释）

demo_drv1.c:

```
#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/mutex.h>
#include <linux/proc_fs.h>
#include <linux/seq_file.h>
#include <linux/uaccess.h>
#include <linux/slab.h>
#define Demo_MAJOR 98          //主设备号
#define VERSION "Demo_Driver"

struct demo_dev{
    int count;
    char buffer[1024];
};

static struct demo_dev *dev_data = NULL;//初始化
static DEFINE_MUTEX(dev_mutex);

static void showversion(void)
{
    printk("*****\n");
    printk("\t%s\t\n", VERSION);
    printk("*****\n\n");
}

int Demo_open(struct inode *inode, struct file *file)
```

```

{
    printk("Demo_open : Enter\n");
    mutex_lock(&dev_mutex);
//dev_data 指针为 NULL, 做初始化
    if(!dev_data){
        dev_data = kmalloc(sizeof(struct demo_dev),GFP_KERNEL);
        if(!dev_data){
            mutex_unlock(&dev_mutex);
            printk("Demo_open : Memory allocation failed\n");
            return -ENOMEM;
        }

        dev_data->count = 1;
        memset(dev_data->buffer,0,sizeof(dev_data->buffer));
        printk("Demo_open : Allocated new device data (count=1)\n");
    }else{
        dev_data->count++;
        printk("Demo_open : Existing device data (count=%d)\n", dev_data->count);
    }
    //重置 f_pos 指针
    file->f_pos = 0;
    printk("Demo_open : Reset file position to 0\n");
    mutex_unlock(&dev_mutex);
    return 0;
}

int Demo_release(struct inode*inode, struct file*file)
{
    printk("Demo_release : Enter\n");

    mutex_lock(&dev_mutex);
    if(dev_data){
        dev_data->count--;
        printk("Demo_release: Decrement count to %d\n", dev_data->count);
        //释放
        if(dev_data->count == 0){
            kfree(dev_data);
            dev_data = NULL;
            printk("Demo_release: Device data freed\n");}
        }

    mutex_unlock(&dev_mutex);
    return 0;}

ssize_t Demo_read(struct file * file, char *buf, size_t count, loff_t *f_pos)
{
    printk("Demo_read : Enter\n");
    size_t bytes_to_read;           //记录要读取的字符数量

```

```

mutex_lock(&dev_mutex);
if(!dev_data){
    printk("Demo_read : Device data not available\n");
    return -ENODEV;}
//防止溢出
bytes_to_read = min(count,sizeof(dev_data->buffer) - *f_pos);

if (bytes_to_read == 0) {
    mutex_unlock(&dev_mutex);
    printk("Demo_read: End of buffer\n");
    return 0; // *f_pos 到达缓冲区末尾
}
//将内容 copy 到用户空间来 read
if (copy_to_user(buf, dev_data->buffer + *f_pos, bytes_to_read)) {
    mutex_unlock(&dev_mutex);
    printk("Demo_read: Failed to copy to user\n");
    return -EFAULT;
}
//将 f_pos 指针移动到已读完的位置，实现在 buffer 中继续向后写的，不覆盖先前内容的功能
*f_pos += bytes_to_read;
printk("Demo_read: Read %zu bytes from offset %lld\n", bytes_to_read, *f_pos);
mutex_unlock(&dev_mutex);
return bytes_to_read;}

ssize_t Demo_write(struct file* file, const char *buf, size_t count, loff_t *f_pos)
{
    printk("Demo_write : Enter\n");
    size_t bytes_to_write;

    if(!dev_data){
        mutex_unlock(&dev_mutex);
        printk("Demo_write : Device data not available\n");
        return -ENODEV;}
//防止写溢出
bytes_to_write = min(count,sizeof(dev_data->buffer) - *f_pos);
if(bytes_to_write == 0){
    mutex_unlock(&dev_mutex);
    printk("Demo_write: Buffer full\n");
    return -ENOSPC;}
//通过函数 copy_from_user 来实现写 buffer
if(copy_from_user(dev_data->buffer + *f_pos,buf,bytes_to_write)){
    mutex_unlock(&dev_mutex);
    printk("Demo_write: Failed to copy from user\n");
    return -EFAULT;}

```

```

//同样的，将指针移动到已经写完的位置，实现向后写的功能
    *f_pos += bytes_to_write;
    printk("Demo_write: Wrote %zu bytes to offset %lld\n", bytes_to_write, *f_pos);
    mutex_unlock(&dev_mutex);
    return bytes_to_write;}

struct file_operations Demo_ops={
    open:      Demo_open,
    read:      Demo_read,
    write:     Demo_write,
    release: Demo_release,};
//模块初始化
static int __init Demo_Init()
{
    int ret = -1;
//注册字符设备
    ret = register_chrdev(Demo_MAJOR,"demo_drv1", &Demo_ops);
    showversion();
    if(ret < 0){
        printk("Demo_module failed with %d\n[--kernel--]", ret);
        return ret;    }
    else{
        printk("Demo_driver register success!!![--kernel--]\n"); }
    return ret;}
//模块卸载
static void __exit Demo_Exit ()
{
    unregister_chrdev(Demo_MAJOR,"demo_drv1");
    mutex_lock(&dev_mutex);
    if(dev_data){
        kfree(dev_data);//释放申请的空间
        dev_data = NULL;//置 NULL
        printk("Demo_Exit: Device data freed\n");
    }
    mutex_unlock(&dev_mutex);
    printk("Demo_driver unregister success!!![--kernel--]\n");}

/*驱动程序版本及 GPL 协议证书信息*/
MODULE_DESCRIPTION("Advanced Character Device Driver with IPC");
MODULE_LICENSE("GPL");

/*内核模块入口,相当于 main()函数,完成模块初始化*/
module_init(Demo_Init);
/*卸载时调用的函数入口,完成模块卸载*/
module_exit(Demo_Exit);

```

writer.c:

```

#include <stdio.h>

```

```

#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>
#include <ctype.h>

#define DEVICE_PATH "/dev/demo_drv1"
#define MAX_MSG_SIZE 1023 // 预留 1 字节给字符串终止符

int main(){

    int fd = open(DEVICE_PATH,O_RDWR);
    if(fd < 0){
        perror("Failed to open device");
        exit(EXIT_FAILURE);}
    printf("Writer: Device opened (FD=%d)\n", fd);

    char buffer[MAX_MSG_SIZE + 1]; // +1 for null-terminator
    char choice = 'y';

    while(1){
        // 获取用户输入
        printf("\n 请输入要写入的消息 (最多%d 字符):\n> ", MAX_MSG_SIZE);
        fflush(stdout); // 确保提示立即显示

        if (!fgets(buffer, sizeof(buffer), stdin)) {
            perror("读取输入失败");
            break;}

        // 移除换行符
        size_t len = strlen(buffer);
        if (len > 0 && buffer[len-1] == '\n') {
            buffer[len-1] = '\0';
            len--;}

        // 处理空输入
        if (len == 0) {
            printf("输入为空，请重新输入\n");
            continue;}

        // 检查退出命令
        if (strcmp(buffer, "exit") == 0 || strcmp(buffer, "quit") == 0) {
            printf("收到退出命令\n");
            break;}

        // 执行写操作
        ssize_t ret = write(fd, buffer, len);

```

```

        if (ret < 0) {
            perror("写入失败");
            //fprintf(stderr, "错误码: %d\n", errno);
        } else {
            printf("成功写入 %zd 字节\n", ret);}

// 询问是否继续
    printf("\n 继续写入? (y/n) ");
    fflush(stdout);
    choice = getchar();
    while (getchar() != '\n'); // 清空输入缓冲区
    if (toupper(choice) != 'Y') {
        break;}}

close(fd);
    printf("Writer: Device closed\n");
return 0;}

```

reader.c:

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define DEVICE_PATH "/dev/demo_drv1"
int main()
{
    int fd = open(DEVICE_PATH, O_RDONLY);
    if (fd < 0) {
        perror("Failed to open device");
        exit(EXIT_FAILURE);}
    printf("Reader: Device opened\n");

    char buffer[1024];
    ssize_t ret = read(fd, buffer, sizeof(buffer)-1);
    if (ret < 0) {
        perror("Read failed");
    } else {
        buffer[ret] = '\0';//sizeof(buffer)-1
        printf("Reader: Read %zd bytes\nMessage: %s\n", ret, buffer);}

    close(fd);
    printf("Reader: Device closed\n");
    return 0;}

```

三、实验结果及其分析

编译成功后，加载驱动程序，创建设备节点：

```

root@ubuntu:linux# cd test1
root@ubuntu:test1# make clean
rm -f *.ko *.mod.c *.o -core.depend
root@ubuntu:test1# rmmod demo_drv1
root@ubuntu:test1# make
make -C /lib/modules/3.2.14/build M=/home/linux/test1 modules
make[1]: 正在进入目录 `/usr/src/linux-source-3.2.0'
  CC [M]  /home/linux/test1/demo_drv1.o
/home/linux/test1/demo_drv1.c: 在函数 'Demo_read' 中:
/home/linux/test1/demo_drv1.c:75: 警告: ISO C90 不允许混合使用声明和代码
/home/linux/test1/demo_drv1.c:83: 警告: 比较不相关的指针时缺少类型转换
/home/linux/test1/demo_drv1.c: 在函数 'Demo_write' 中:
/home/linux/test1/demo_drv1.c:107: 警告: ISO C90 不允许混合使用声明和代码
/home/linux/test1/demo_drv1.c:115: 警告: 比较不相关的指针时缺少类型转换
/home/linux/test1/demo_drv1.c: 在文件层:
/home/linux/test1/demo_drv1.c:143: 警告: 函数声明不是一个原型
/home/linux/test1/demo_drv1.c:158: 警告: 函数声明不是一个原型
Building modules, stage 2.
MODPOST 1 modules
  CC      /home/linux/test1/demo_drv1.mod.o
  LD [M]  /home/linux/test1/demo_drv1.ko
make[1]:正在离开目录 `/usr/src/linux-source-3.2.0'
root@ubuntu:test1# insmod demo_drv1.ko
root@ubuntu:test1# mknod /dev/demo_drv1 c 98 0

```

通过 writer 来写 buffer，首先写入 Yao Yanan，再写入 98989898，最后终止进程：

```

root@ubuntu:linux# cd test1
root@ubuntu:test1# gcc writer.c -o writer
root@ubuntu:test1# ./writer
Writer: Device opened (FD=3)

请输入要写入的消息 (最多1023字符):
> Yao Yanan
成功写入 9 字节

继续写入? (y/n) y

请输入要写入的消息 (最多1023字符):
> 98989898
成功写入 8 字节

继续写入? (y/n) n
Writer: Device closed
root@ubuntu:test1#

```

通过 reader 来读 buffer（两个不同的进程实现进程间通信），第一次读到第一次写入的内容：Yao Yaonan；第二次读到 Yao Yanan98989898；最后当写 buffer 的进程终止后，按照 demo_drv1.c 的逻辑，count 减为 0，则释放 open 中申请的空间，并把指针置为 NULL，此时 reader 读到的为空的 buffer


```

linux@ubuntu:~$ cd test1
linux@ubuntu:test1$ gcc reader.c -o read
linux@ubuntu:test1$ ./reader
Reader: Device opened
Reader: Read 1023 bytes
Message: Yao Yanan
Reader: Device closed
linux@ubuntu:test1$ ./reader
Reader: Device opened
Reader: Read 1023 bytes
Message: Yao Yanan98989898
Reader: Device closed
linux@ubuntu:test1$ ./reader
Reader: Device opened
Reader: Read 1023 bytes
Message:
Reader: Device closed
linux@ubuntu:test1$

```

最后是删除设备节点，卸载驱动程序和详细打印的信息：

```

root@ubuntu:test1# rm /dev/demo_drv1
root@ubuntu:test1# rmmmod demo_drv1

```

```

[ 813.439797] *****
[ 813.439800] Demo_Driver
[ 813.439802] *****
[ 813.439802]
[ 813.439803] Demo_driver register success!!![--kernel--]
[ 947.368947] Demo_open : Enter
[ 947.368951] Demo_open : Allocated new device data (count=1)
[ 947.368953] Demo_open : Reset file position to 0
[ 957.578329] Demo_write : Enter
[ 957.578334] Demo_write: Wrote 9 bytes to offset 9
[ 1051.478124] Demo_open : Enter
[ 1051.478129] Demo_open : Existing device data (count=2)
[ 1051.478131] Demo_open : Reset file position to 0
[ 1051.478186] Demo_read : Enter
[ 1051.478189] Demo_read: Read 1023 bytes from offset 1023
[ 1051.478235] Demo_release : Enter
[ 1051.478237] Demo_release: Decrement count to 1
[ 1065.539575] Demo_write : Enter
[ 1065.539580] Demo_write: Wrote 8 bytes to offset 17
[ 1068.940127] Demo_open : Enter
[ 1068.940131] Demo_open : Existing device data (count=2)
[ 1068.940133] Demo_open : Reset file position to 0
[ 1068.940170] Demo_read : Enter
[ 1068.940172] Demo_read: Read 1023 bytes from offset 1023
[ 1068.940202] Demo_release : Enter
[ 1068.940204] Demo_release: Decrement count to 1
[ 1073.415812] Demo_release : Enter
[ 1073.415815] Demo_release: Decrement count to 0
[ 1073.415816] Demo_release: Device data freed
[ 1079.984376] Demo_open : Enter
[ 1079.984380] Demo_open : Allocated new device data (count=1)
[ 1079.984382] Demo_open : Reset file position to 0
[ 1079.984420] Demo_read : Enter
[ 1079.984423] Demo_read: Read 1023 bytes from offset 1023
[ 1079.984452] Demo_release : Enter
[ 1079.984454] Demo_release: Decrement count to 0
[ 1079.984456] Demo_release: Device data freed
[ 1176.898286] Demo_driver unregister success!!![--kernel--]

```